

Reliable Curation of EHR Dataset via Large Language Models under Environmental Constraints

Raymond M. Xiong

*Department of Computer Science
Duke University
Durham, NC, USA
raymond.xiong@duke.edu*

Panyu Chen

*Department of Computer Science
Duke University
Durham, NC, USA
panyu.chen@duke.edu*

Tianze Dong

*Department of Computer Science
Duke University
Durham, NC, USA
tianze.dong@duke.edu*

Jian Lu

*Department of Computer Science
Duke University
Durham, NC, USA
jian.lu@duke.edu*

Louis Hu

*Department of Computer Science
Duke University
Durham, NC, USA
louis.hu@duke.edu*

Nathan Yu

*Department of Computer Science
Duke University
Durham, NC, USA
nathan.yu@duke.edu*

Benjamin Goldstein

*Department of Biostatistics &
Bioinformatics
Duke University
Durham, NC, USA
ben.goldstein@duke.edu*

Danyang Zhuo

*Department of Computer Science
Duke University
Durham, NC, USA
danyang.zhuo@duke.edu*

Anru R. Zhang

*Department of Biostatistics &
Bioinformatics
Department of Computer Science
Duke University
Durham, NC, USA
anru.zhang@duke.edu*

Abstract—Electronic health records (EHRs) are central to modern healthcare delivery and research; yet, many researchers lack the database expertise necessary to write complex SQL queries or generate effective visualizations, limiting efficient data use and scientific discovery. To address this barrier, we introduce CELEC, a large language model (LLM)-powered framework for automated EHR data extraction and analytics. CELEC translates natural language queries into SQL using a prompting strategy that integrates schema information, few-shot demonstrations, and chain-of-thought reasoning, which together improve accuracy and robustness. CELEC also adheres to strict privacy protocols: the LLM accesses only database metadata (e.g., table and column names), while all query execution occurs securely within the institutional environment, ensuring that no patient-level data is ever transmitted to or shared with the LLM. On a subset of the EHRSQL benchmark, CELEC achieves execution accuracy comparable to prior systems while maintaining low latency, cost efficiency, and strict privacy by exposing only database metadata to the LLM. Ablation studies confirm that each component of the SQL generation pipeline, particularly the few-shot demonstrations, plays a critical role in performance. By lowering technical barriers and enabling medical researchers to query EHR databases directly, CELEC streamlines research workflows and accelerates biomedical discovery.

Index Terms—Electronic Health Records, Automated Data Analytics, Large Language Models, Text-to-SQL

I. INTRODUCTION

Electronic health records (EHRs) have become a central component of modern healthcare. Since 2009, EHR adoption has increased tenfold, and by 2020, more than 90% of primary care providers reported daily use [1], [2]. When effectively

implemented, EHRs improve the accuracy and continuity of patient care while enabling large-scale, cost-efficient data analysis that supports both clinical research and population health management [3]–[5]. Unlocking this potential requires methods that make EHR data accessible and interpretable for research, advancing both scientific discovery and healthcare systems.

In practice, however, EHR systems remain challenging for researchers to use. Extracting meaningful data often requires navigating fragmented interfaces and applying technical skills such as writing structured queries, which most researchers lack formal training in [4], [6]. These barriers limit exploratory analysis and necessitate reliance on database specialists, leaving valuable research opportunities underutilized. Tools that allow researchers to access EHR data and conduct basic analytics without programming expertise could therefore lower entry barriers and accelerate biomedical discovery [7].

Prior work has attempted to bridge this gap using text-to-SQL methods. While these approaches have shown strong performance on EHR question-answering benchmarks [8]–[11], it is often unclear whether the underlying database systems are exposed to cloud-based LLMs, which can be a significant risk for sensitive medical datasets, where data leakage might compromise patient privacy.

To address the security concern, we propose **CELEC (Curation of EHR via LLMs under Environmental Constraints)**, a system that restricts LLM access to schema-level metadata only and executes all SQL queries locally,

ensuring that no patient-level data is transmitted or shared inadvertently with external services. As a framework for automated EHR data extraction and analytics, CELEC translates natural-language (NL) questions into SQL using a prompting strategy that combines schema information, few-shot demonstrations, and chain-of-thought reasoning to generate accurate SQL outputs. Once data are retrieved, CELEC also generates simple visualizations directly from extracted dataframes, providing immediate exploratory insights. Designed with real-world constraints in mind, CELEC maintains low latency and cost efficiency. By lowering technical barriers, CELEC enables researchers to query EHR databases and perform preliminary analyses independently, thereby streamlining workflows and accelerating discovery.

II. RELATED WORK

A. EHR system usability & data accessibility

Despite near-universal adoption of EHRs [1], [12], effective use remains limited by fragmented information, disrupted workflows, and high cognitive load, which are commonly linked to errors, inefficiency, and burnout [13], [14]. Prior research has shown that extensions such as custom dashboards and enhanced visualization of clinical trends can improve usability [15], [16]. More recently, automated visualization approaches have been explored in the medical domain [17]; however, these efforts often focused on small-scale applications and electronic medical records rather than large-scale, multi-table EHR systems.

Beyond usability in care delivery, EHR data is also essential for research, quality improvement, and population health management. Nevertheless, accessibility remains a major barrier, as healthcare professionals often lack the training required to manipulate complex database systems [4], [6]. To reduce this barrier, several interactive EHR query tools have been proposed. For example, i2b2 [18] provides a cohort builder that allows users to construct inclusion and exclusion criteria using a hierarchical ontology of clinical concepts. Subsequent systems, such as LEAF [19] and OMOP/ATLAS [20], follow a similar paradigm, augmenting it with modern web interfaces and richer configuration options to make the experience more intuitive. While these tools eliminate the need to write SQL explicitly, they still require users to think in terms of database structure and to formulate queries using controlled vocabularies and formal logic, ultimately limiting accessibility for users with minimal exposure to databases and programming.

In contrast, CELEC removes the need for users to understand query logic, database structure, or any programming language. Instead, users simply pose a natural-language question, and the system automatically retrieves relevant data, performs the necessary analysis, and produces both tabular results and visual summaries. By unifying natural language querying with flexible analytics and visualization within a single pipeline, CELEC provides a deployment-ready framework that directly supports research-driven use cases. This question-driven interaction paradigm offers a substantially more intuitive experience

than schema-aware query builders and fundamentally distinguishes CELEC from existing interactive EHR tools.

B. Medical text-to-SQL

General-purpose text-to-SQL models have achieved strong results on benchmarks such as Spider [21] and WikiSQL [22] [23]–[25]. However, performance on these datasets does not guarantee success in clinical contexts, where data complexity and privacy requirements differ substantially from business applications. Early medical-domain systems relied on templates for text-to-SQL translation, which limited their ability to address complex real-world questions [26]. The EHRSQL shared task [27], [28] advanced the field by aligning hospital-generated natural language queries with major EHR databases such as MIMIC-III, eICU, and MIMIC-IV demo. Leading teams on the benchmark explored methods including schema-aware models [8], ensemble prompting [9], and probability-based SQL verification [10]. CELEC employs a streamlined two-call design that balances accuracy and low latency while strictly adhering to privacy protocols. Relevant quantitative evaluations are presented in Section IV.

C. LLMs in Healthcare

Most LLM applications in healthcare focused on decision support and conversational interfaces for clinicians [29], [30]. However, little attention has been given to applications that assist researchers with structured data analytics. Recent work has explored connecting EHR databases with Claude Desktop for LLM-powered analytics [11]. In contrast, CELEC introduces a privacy-conscious design that restricts LLM inputs to schema-level metadata, thereby minimizing exposure of raw patient data and reducing the attack surface for leakage.

III. METHODS

We designed the CELEC system¹ as an end-to-end application (see Figure 1). In our implementation, the system connects to DuckDB versions of MIMIC-III [31] and MIMIC-IV [32] databases, which are large EHR databases comprising de-identified health-related data, including demographics, measurements, laboratory test results, procedures, and medications. Users can enter a natural language (NL) question to describe their analysis objective. Then, an LLM translates the NL question to an SQL query that extracts a succinct yet necessary set of data to answer it. We use prompt engineering to enhance the quality of generated SQL code (Section III-A). Next, we connect to the EHR database and execute the generated SQL to retrieve the corresponding data, after which an LLM creates a visualization to help users better explore their data (Section III-B).

A. SQL generation

We use the o3-2025-04-16 model to generate SQL queries from NL input. To improve reliability and alignment with the target EHR database, our prompt design incorporates three key elements: (1) **schema information**, (2) **few-shot**

¹The source code and data will be released through GitHub.

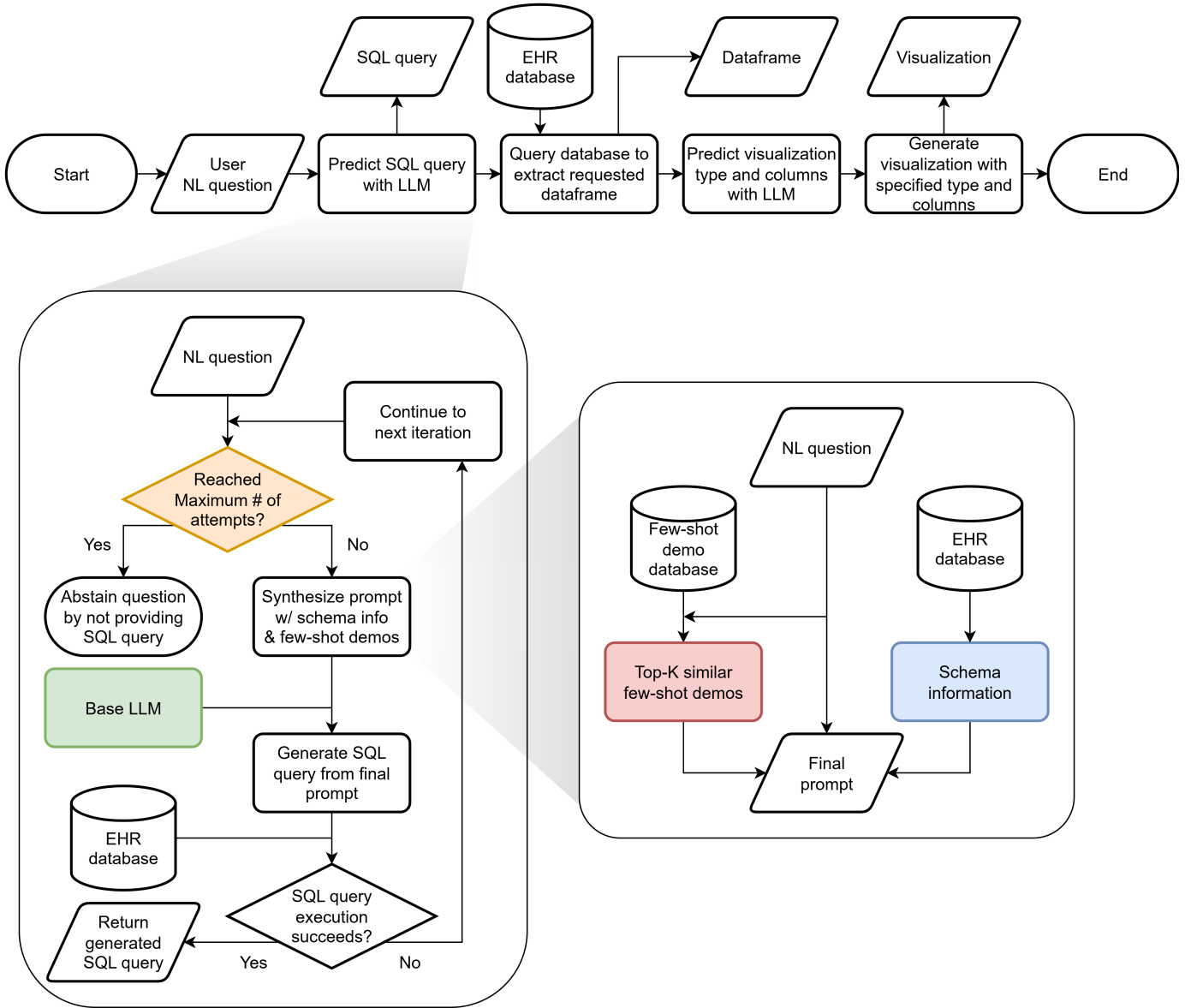


Fig. 1. Overview of the CELEC system design.

demonstrations, and (3) **chain-of-thought (CoT)** reasoning. The complete prompt template is provided in Appendix A. Importantly, only schema-level metadata (table names, column names, and column types) are passed to the LLM; no patient-level data are ever exposed to the LLM.

Schema information. For each database, schema metadata are embedded in the prompt to inform the LLM of available tables and attributes, which reduces hallucination and prevents fabrication of nonexistent aliases. Each table is represented by its columns, types, and constraints. More details may be found in Appendix A.

Few-shot demonstrations. We include in-context few-shot demonstrations of NL questions paired with their corresponding SQL queries. Demonstrations come from two sources: (a) medical literature using MIMIC datasets, where questions were adapted from published cohort criteria and manually verified (105 demos; see Appendix B), and (b) the EHRSQL

benchmark [28], from which we preprocessed the training and validation sets into 4,761 high-quality pairs (see Appendix C). These sources are combined into a single demo database; during inference, CELEC selects the top- k most similar demos ($k = 2$) based on cosine similarity between the input question’s embedding and the embeddings of the demo questions. Embeddings are computed using `all-MiniLM-L6-v2` and indexed with ChromaDB for efficient retrieval.

Chain-of-thought (CoT). To guide complex query construction, we augment the prompt with an intermediate reasoning step where the LLM first identifies potentially relevant tables before generating the final SQL. This step enhances alignment between the input question and the selected schema elements, thereby reducing unnecessary joins and extraneous columns. Crucially, the few-shot demonstrations are also formatted to include this intermediate reasoning. By showing demos that explicitly map NL questions to table-selection steps

and then to final SQL, we encourage the model to follow the same structure. This not only enhances interpretability but also stabilizes SQL generation in complex queries.

Error handling. After SQL generation, the query is executed locally on the target database. If execution fails (typically due to hallucinations of aliases or function names), the error message is appended to the prompt, and the LLM retries the generation. By default, up to two retries are allowed, which balances robustness with latency (see ablation results in Section V). Error messages are processed only by the LLM during retries and are not exposed to users.

B. Visualization generation

As many research and clinical workflows benefit from visual summaries that facilitate pattern interpretations, CELEC also includes an LLM-powered visualization module. After the SQL query is executed and a dataframe retrieved, we provide the LLM with the input NL question and the column metadata from the resulting table. The model predicts (1) an appropriate visualization type (e.g., histogram, bar chart, line plot, scatterplot) and (2) the aesthetic mappings, such as which variables should be plotted on the horizontal and vertical axes.

The prompt includes both instructions and simple input-output examples (see Appendix A), encouraging the model to output a structured specification. Importantly, as in SQL generation, only schema-level metadata is exposed to the LLM. Once the visualization specification is returned, CELEC renders the chart using a set of hard-coded TextScript functions. This design ensures consistency and robustness across visualizations, while protecting privacy and minimizing LLM-induced errors. By integrating visualization generation, CELEC enables users to move beyond raw tabular results and obtain exploratory insights directly, thereby lowering the barriers for both researchers and clinicians.

C. System integration

While Sections III-A and III-B describe SQL and visualization generation individually, their integration is key to CELEC’s usability in real-world research settings. The modules are orchestrated in a unified pipeline: SQL queries are generated and executed locally, and results can either be returned directly or forwarded to the visualization module for immediate exploratory analysis.

This integration offers system-level benefits that neither module alone captures. First, the streamlined design, which relies on only two LLM calls per query, reduces latency compared to ensemble or verification-heavy approaches. Second, the privacy-conscious architecture ensures that at no stage do LLMs access patient-level data; only schema metadata and aggregate specifications are processed. Finally, presenting SQL and visualization under a single framework reduces the technical barrier for researchers and clinicians alike, supporting both reproducible research and practical deployment.

IV. EVALUATION

We evaluate CELEC on the EHRSQL-2024 benchmark [28], which contains more than 7,000 pairs of NL questions and

gold SQL queries against a modified version of the MIMIC-IV demo database. The benchmark is designed to reflect real-world clinical needs, with complex queries typically involving temporal conditions, group operations, and cross-table joins.

A. Dataset adaptation

To align the benchmark with CELEC, we filtered and modified the provided train, validation, and test splits. In particular, we removed unanswerable questions where the gold SQL did not execute successfully on the MIMIC-IV demo database (see Appendix C). After adaptation, we used all training and validation points as few-shot candidates for SQL prompting, 10% of the new test set (78 queries) as validation data for hyperparameter tuning, and the remaining 90% (707 queries) for evaluation. By ensuring that all test queries are executable, this adaptation yields a more reliable measure of model capability. We note that the deviation from the official leaderboard split precludes direct performance comparisons between CELEC and published systems; we present the leaderboard numbers as a contextual reference for understanding CELEC’s performance.

B. Evaluation methods

We evaluate system performance using the RS(0) score, the official metric of the EHRSQL benchmark. RS(0) extends execution accuracy (EX) by rewarding abstention on unanswerable questions. Formally, if X is the set of all questions and $X_{ans} \subseteq X$ the subset of answerable ones, then

$$RS(0) = \frac{1}{|X|} \sum_{x \in X} \mathbb{1} [(x \in X_{ans} \wedge R_{gen}(x) = R_{gt}(x)) \vee (x \notin X_{ans} \wedge g(x) = 0)].$$

Since our adapted test set excludes unanswerable questions ($X_{ans} = X$), RS(0) reduces to execution accuracy in our evaluation, but we report it as RS(0) for consistency with leaderboard scores.

TABLE I
COMPARISON OF CELEC WITH TEAMS FROM THE EHRSQL-2024 LEADERBOARD.

System / Team	RS(0) (%)
CELEC (Ours)	81.05
LG AI Research & KAIST [8]	88.17
PromptMind [9]	82.60
ProbGate [10]	81.92
GPT-4o [33]	76.45
KU-DMIS [34]	72.07
AIRI NLP [35]	68.89
MedCopilot [33]	68.78
LTRC-IIITH [36]	66.84
Text2SQL-Flow [37]	58.70
Saama Technologies [38]	53.21

C. Results & Discussion

Table I compares CELEC with leading systems from the EHRSQL-2024 leaderboard. On our adapted test set, CELEC achieves 81.05% RS(0) accuracy, comparable to existing systems, while maintaining strong privacy guarantees. Ablation

TABLE II
RS(0) SCORE OF CELEC UNDER DIFFERENT PARAMETER SETTINGS. THE FIRST ROW REPRESENTS OUR DEFAULT SETTINGS.

Base LLM	Schema info? (Y/N)	# of few-shot demos	Max # of attempts	RS(0) (%)
o3	Y	2	2	81.05
o4-mini gpt-4.1	Y	2	2	73.41
	Y	2	2	77.93
o3	N	2	2	77.93
o3	Y	1	2	73.97
o3	Y	0	2	50.21
o3	Y	2	1	79.21

studies in Section V further confirm that few-shot prompting and retry mechanisms are critical to this performance.

In terms of latency, CELEC processes one NL query in an average of 6.02 to 6.11 seconds across the test set. As for API costs, CELEC incurs an average cost of \$0.0152 per question across the test set. While other systems have not disclosed inference times and associated costs, CELEC’s streamlined two-call design demonstrates practical feasibility for interactive use and large-scale deployment.

Finally, to safeguard privacy, we verified automatically that all generated SQL queries only reference schema-level columns and tables. No queries attempted to access fabricated or patient-level identifiers, consistent with CELEC’s metadata-only exposure design.

V. ABLATION STUDY

We conducted ablation studies to assess the contribution of individual system components. In each experiment, we modified one component while keeping all others fixed, and report the RS(0) score on the adapted EHRSQL test set (Table II). Results consistently show that each component makes a meaningful contribution to performance, with in-context learning yielding the largest gains.

Base LLM. The choice of the base model had a great impact on accuracy. Our default model, o3-2025-04-16, achieved the highest score of 81.05%. Replacing it with the smaller reasoning model o4-mini-2025-04-16 reduced accuracy to 73.41%, whereas using gpt-4.1-2025-04-14, a non-reasoning model, yielded 77.93%. This suggests that both model scale and reasoning optimization are essential for handling the complex queries in EHRSQL.

Schema information. Providing schema metadata to the LLM significantly improved reliability. Without schema information, performance dropped from 81.05% to 77.93%, with frequent hallucinations of nonexistent columns or aliases. This confirms that grounding query generation in explicit schema details is critical to reducing structural SQL errors.

Few-shot demonstrations. In-context demonstrations had the most significant impact of all components. Without demonstrations (zero-shot), CELEC achieved only 50.21%. Adding a single demonstration increased performance to 73.97%, and using two demonstrations, which is our default, further

improved it to 81.05%. This trend underscores the crucial role of in-context learning in aligning the model with the structure and semantics of medical databases, with diminishing returns evident beyond two examples.

Maximum attempts. Allowing the model to retry once after a failed SQL execution improved accuracy from 79.21% (single attempt) to 81.05% (two attempts). Error analysis indicates that many first-pass failures stemmed from superficial issues, such as hallucinations of aliases or function names. Providing the database error message during a second attempt often corrected these, yielding modest but consistent gains with minimal latency overhead.

Overall, these results demonstrate that each design choice contributes to CELEC’s performance, with few-shot demonstrations being the most critical factor. The combination of schema grounding, a reasoning-capable base model, and a lightweight retry mechanism together supports robust SQL generation in complex EHR environments.

VI. ERROR MODE ANALYSIS

To better understand the patterns underlying failed queries, we conducted an error mode analysis. We manually inspected cases where the result table produced by the predicted SQL did not match the ground-truth output and categorized the observed failures. Several recurring error modes emerged, offering insights into the model’s limitations and suggesting concrete directions for future improvement (Figure 2).

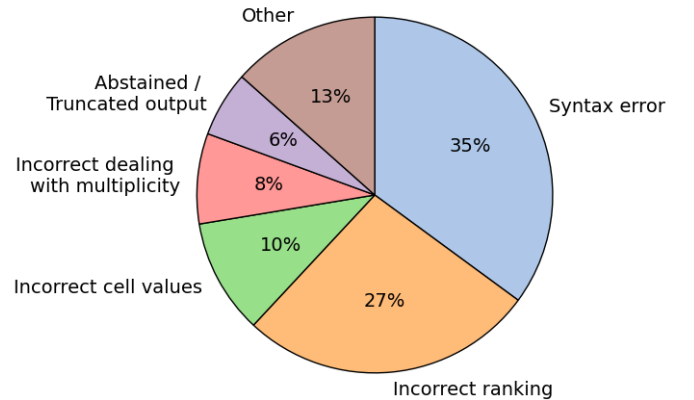


Fig. 2. Statistics of failure modes under default settings.

Syntax error. Syntax errors were the most common failure mode (35% of failures). These errors typically involved undefined functions or invalid function arguments, often reflecting confusion between DuckDB and more traditional SQL dialects. Future work in general-domain Text-to-SQL could consider more explicit modeling of SQL dialect variation, potentially by developing new benchmarks.

Incorrect ranking. Incorrect ranking was the second most common error type (27% of failures). These errors typically occurred for questions that requested the top K results. Ground-truth SQL queries often use the `DENSE_RANK` function, allowing the output to include more than K rows when ties occur at the cutoff, whereas the predicted SQL queries frequently use `ORDER_BY` with `LIMIT`, returning exactly K rows. This discrepancy suggests a limitation of the system in handling multiple valid interpretations of NL ranking questions. Future work could explore uncertainty-aware decoding or multi-turn interaction mechanisms that allow the system to request clarification when such ambiguities arise.

Incorrect cell values. In 10% of failures, the predicted SQL queries referenced incorrect cell values due to exact string matching, which often resulted in an empty table. Typical examples include mismatches in label specificity or casing (e.g., “height” vs “height (cm)” for patient height, “M” vs “m” for male gender). These failures also highlight the model’s difficulty in handling uncertainty and subtle mismatches between natural language and database-specific values. Incorporating mechanisms such as fuzzy string matching or value normalization could help improve robustness and reliability.

Incorrect dealing with multiplicity. Another observed failure mode concerns the handling of multiplicity in query answers. In some cases, the predicted SQL used a subquery that returned multiple rows in a context that expected a scalar value, which results in an invalid query. In other cases, the NL question permits multiple valid rows in the result table, but the predicted query incorrectly restricts the output to a single row.

Abstained or truncated output. The system also abstained from answering or produced truncated outputs for answerable questions, accounting for 6% of the failure cases.

Miscellaneous. Other less frequent error modes include incorrect output formats, incorrect table or column names, and arithmetic or aggregation errors.

Overall, our error analysis shows that failure cases in CELEC Text-to-SQL generation exhibit consistent and interpretable patterns. These patterns reveal the model’s limitations in adapting to emergent SQL dialects and handling ambiguity in NL queries, suggesting directions for future work.

VII. DISCUSSION & CONCLUSION

We introduced CELEC, a large language model-powered privacy-constrained framework for automated EHR data extraction and analytics. CELEC translates natural language questions into executable SQL queries and can generate simple visualizations of query results, lowering the expertise required to access and explore EHR data. Importantly, CELEC adheres to strict privacy protocols: the LLM accesses only

database metadata and does not access patient-level data. In an evaluation on answerable MIMIC-IV cases in the EHRSQL-2024 benchmark, CELEC achieved an RS(0) score of 81.05%, comparable to existing systems, while adhering to a privacy-conscious design and maintaining low latency and low cost. Our ablation studies further highlighted the critical role of few-shot demonstrations, schema grounding, reasoning-optimized base models, and lightweight retry mechanisms in achieving robust SQL generation. Together, these results demonstrate that CELEC is an effective and practical approach to curating easily accessible EHR datasets. It has high potential to bridge the accessibility gap between researchers and large-scale EHR databases, streamline workflows and accelerate discovery in healthcare research.

Several limitations of our work suggest directions for future research. First, our evaluation has limitations in both benchmarking and component-level assessment. To ensure executability, we filtered out unanswerable EHRSQL queries, which prevents direct comparison with systems evaluated on the original benchmark splits; additionally, we do not evaluate the visualization module and provide quantitative metrics to assess the correctness or appropriateness of the generated visualizations. Developing standardized evaluation criteria for data visualization in text-to-SQL pipelines is an important direction for future work. Second, our system is evaluated exclusively on the MIMIC database schema. While our system is not architecturally tied to MIMIC-specific table structures and terminology, further validation on additional EHR databases would help assess CELEC’s robustness and portability across different settings. Finally, we do not conduct real-world deployment evaluations. While the system is designed to lower technical barriers for clinical researchers, future user studies would help determine its practical utility, usability, and impact on research workflows.

ACKNOWLEDGMENT

We thank anonymous IEEE ICHI reviewers for their constructive feedback on our paper. The research of A. R. Zhang was supported in part by NIH Grant R01HL169347. R. M. Xiong acknowledges support from the Duke CS+ summer research program.

REFERENCES

- [1] W. Barker, W. Chang, J. Everson, M. Gabriel, V. Patel, C. Richwine, and C. Strawley, “The evolution of health information technology for enhanced patient-centric care in the united states: Data-driven descriptive study,” *J Med Internet Res*, vol. 26, p. e59791, Oct 2024.
- [2] G. Kerr, N. Kulshreshtha, G. Greenfield, E. Li, T. Beaney, B. Hayhoe, J. Car, A. Clavería, C. Collins, S. Espitia, M. Fernandez, G. Gusso, K. Hoedebecke, R. Hoffman, G. Irving, G. Jimenez, L. Laranjo, V. Lazić, H. Lingner, E. Memarian, K. Nessler, B. O’Neill, D. Petek, A. Serafini, M. Ugan, A. Majeed, and A. Neves, “Features and frequency of use of electronic health records in primary care across 20 countries: a cross-sectional study,” *Public Health*, vol. 233, pp. 45–53, 2024.
- [3] J. A. Casey, B. S. Schwartz, W. F. Stewart, and N. E. Adler, “Electronic Health Records and Population Health Research,” *Front Public Health Serv Sys Res*, 2016.
- [4] C. S. Kruse, A. Stein, H. Thomas, and H. Kaur, “The use of Electronic Health Records to Support Population Health: A Systematic Review of the Literature,” *J Med Syst*, vol. 42, p. 214, Nov. 2018.

- [5] J. Chishtie, N. Sapiro, N. Wiebe, L. Rabatach, D. Lorenzetti, A. A. Leung, D. Rabi, H. Quan, and C. A. Eastwood, "Use of Epic Electronic Health Record System for Health Care Research: Scoping Review," *J Med Internet Res*, vol. 25, p. e51003, Dec. 2023.
- [6] C. H. Tsai, A. Eghdam, N. Davoody, G. Wright, S. Flowerday, and S. Koch, "Effects of electronic health record implementation and barriers to adoption and use: A scoping review and qualitative analysis of the content," *Life*, vol. 10, no. 12, 2020.
- [7] K. Honeyford, P. Expert, E. Mendelsohn, B. Post, A. Faisal, B. Glampson, E. Mayer, and C. Costelloe, "Challenges and recommendations for high quality research using electronic health records," *Frontiers in Digital Health*, vol. 4, 2022.
- [8] Y. Jo, S. Lee, M. Seo, S. J. Hwang, and M. Lee, "LG AI research & KAIST at EHRSQL 2024: Self-training large language models with pseudo-labeled unanswerable questions for a reliable text-to-SQL system on EHRs," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 635–643, Association for Computational Linguistics, June 2024.
- [9] S. Gundabathula and S. Kolar, "PromptMind team at EHRSQL-2024: Improving reliability of SQL generation using ensemble LLMs," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 360–366, Association for Computational Linguistics, June 2024.
- [10] S. Kim, D. Han, and S. Kim, "ProbGate at EHRSQL 2024: Enhancing SQL query generation accuracy through probabilistic threshold filtering and error handling," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 687–696, Association for Computational Linguistics, June 2024.
- [11] R. A. Attrach, P. Moreira, R. Fani, R. Umeton, and L. A. Celi, "Conversational llms simplify secure clinical data access, understanding, and analysis," *arXiv preprint arXiv:2507.01053*, 2025.
- [12] Y. Shen, J. Yu, J. Zhou, and G. Hu, "Twenty-Five Years of Evolution and Hurdles in Electronic Health Records and Interoperability in Medical Research: Comprehensive Review," *J Med Internet Res*, vol. 27, p. e59024, Jan. 2025.
- [13] E. Asgari, J. Kaur, G. Nuredini, J. Balloch, A. M. Taylor, N. Sebire, R. Robinson, C. Peters, S. Sridharan, and D. Pimenta, "Impact of Electronic Health Record Use on Cognitive Load and Burnout Among Clinicians: Narrative Review," *JMIR Med Inform*, vol. 12, p. e55499, Apr. 2024.
- [14] O. Olakotan, R. Samuriwo, H. Ismaila, and S. Atiku, "Usability Challenges in Electronic Health Records: Impact on Documentation Burden and Clinical Workflow: A Scoping Review," *Evaluation Clinical Practice*, vol. 31, p. e70189, June 2025.
- [15] L. M. Mazur, P. R. Mosaly, C. Moore, and L. Marks, "Association of the Usability of Electronic Health Records With Cognitive Workload and Performance Levels Among Physicians," *JAMA Netw Open*, vol. 2, p. e191709, Apr. 2019.
- [16] A. H. Pollack and W. Pratt, "Association of Health Record Visualizations With Physicians' Cognitive Load When Prioritizing Hospitalized Patients," *JAMA Netw Open*, vol. 3, p. e1919301, Jan. 2020.
- [17] H. Zhang, S. Ning, Q. Zheng, J. Nie, L. Zhang, W. Wang, and Y. Song, "Towards visualizing electronic medical records via natural language queries," 2025.
- [18] S. N. Murphy, G. Weber, M. Mendis, V. Gainer, H. C. Chueh, S. Churchill, and I. Kohane, "Serving the enterprise and beyond with informatics for integrating biology and the bedside (i2b2)," *Journal of the American Medical Informatics Association*, vol. 17, no. 2, pp. 124–130, 2010.
- [19] N. J. Dobbins, C. H. Spital, C. M. Black, T. B. Morrison, T. de Silva, A. Roussos, and T. G. Kannampallil, "Leaf: a lightweight self-service web application for querying clinical data from heterogeneous data models and sources," *Journal of the American Medical Informatics Association*, vol. 27, no. 1, pp. 109–117, 2020.
- [20] G. Hripcsak, J. D. Duke, N. H. Shah, C. G. Reich, V. Huser, M. J. Schuemie, M. A. Suchard, R. W. Park, I. C. K. Wong, P. R. Rijnbeek, et al., "Observational health data sciences and informatics (ohdsi): Opportunities for observational researchers," *Studies in Health Technology and Informatics*, vol. 216, pp. 574–578, 2015.
- [21] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," 2019.
- [22] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," 2017.
- [23] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," 2023.
- [24] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," 2023.
- [25] Z. Li, X. Wang, J. Zhao, S. Yang, G. Du, X. Hu, B. Zhang, Y. Ye, Z. Li, R. Zhao, and H. Mao, "Pet-sql: A prompt-enhanced two-round refinement of text-to-sql with cross-consistency," 2024.
- [26] P. Wang, T. Shi, and C. K. Reddy, "Text-to-sql generation for question answering on electronic medical records," 2020.
- [27] G. Lee, H. Hwang, S. Bae, Y. Kwon, W. Shin, S. Yang, M. Seo, J.-Y. Kim, and E. Choi, "Ehsql: A practical text-to-sql benchmark for electronic health records," 2023.
- [28] G. Lee, S. Kweon, S. Bae, and E. Choi, "Overview of the EHRSQL 2024 shared task on reliable text-to-SQL modeling on electronic health records," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 644–654, Association for Computational Linguistics, June 2024.
- [29] D. Oniani, X. Wu, S. Visweswaran, S. Kapoor, S. Kooragayalu, K. Polanska, and Y. Wang, "Enhancing large language models for clinical decision support by incorporating clinical practice guidelines," 2024.
- [30] N. C. Rajashekar, Y. E. Shin, Y. Pu, S. Chung, K. You, M. Giuffre, C. E. Chan, T. Saarinen, A. Hsiao, J. Sekhon, A. H. Wong, L. V. Evans, R. F. Kizilcec, L. Laine, T. McCall, and D. Shung, "Human-algorithmic interaction using a large language model-augmented artificial intelligence clinical decision support system," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, (New York, NY, USA), Association for Computing Machinery, 2024.
- [31] A. Johnson, T. Pollard, and R. Mark, "Mimic-iii clinical database (version 1.4)," *PhysioNet*, vol. 10, no. C2XW26, p. 2, 2016.
- [32] A. Johnson, L. Bulgarelli, T. Pollard, B. Gow, B. Moody, S. Horng, L. Celi, and R. Mark, "Mimic-iv (version 3.1), 2024," URL <https://doi.org/10.13026/kpb9-mt58>.
- [33] R. Xu, Y. Zhuang, Y. Zhong, Y. Yu, Z. Wang, X. Tang, H. Wu, M. D. Wang, P. Ruan, D. Yang, T. Wang, G. Xiao, X. Liu, C. Yang, Y. Xie, and W. Shi, "MedAgentGym: A Scalable Agentic Training Environment for Code-Centric Reasoning in Biomedical Data Science," 2025.
- [34] S. Park, H. Kim, Y. Kim, M. Jeong, M. Sung, E. Choi, and J. Lee, "KU-DMIS at EHRSQL 2024: Generating SQL query via question templization in EHR," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 462–468, Association for Computational Linguistics, June 2024.
- [35] O. Somov, A. Dontsov, and E. Tutubalina, "AIRI NLP team at EHRSQL 2024 shared task: T5 and logistic regression to the rescue," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 431–438, Association for Computational Linguistics, June 2024.
- [36] J. Thomas, P. Mishra, D. Sharma, and P. Krishnamurthy, "LTRC-IIITH at EHRSQL 2024: Enhancing reliability of text-to-SQL systems through abstention and confidence thresholding," in *Proceedings of the 6th Clinical Natural Language Processing Workshop (ClinicalNLP)* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 697–702, Association for Computational Linguistics, June 2024.
- [37] Q. Cai, H. Liang, C. Xu, T. Xie, W. Zhang, and B. Cui, "Text2SQL-Flow: A Robust SQL-Aware Data Augmentation Framework for Text-to-SQL," 2025.
- [38] M. Jabir, K. Kanakarajan, and M. Sankarasubbu, "Saama technologies at EHRSQL 2024: SQL generation through classification answer selector by LLM," in *Proceedings of the 6th Clinical Natural Language Processing Workshop* (T. Naumann, A. Ben Abacha, S. Bethard, K. Roberts, and D. Bitterman, eds.), (Mexico City, Mexico), pp. 655–671, Association for Computational Linguistics, June 2024.

APPENDIX A

PROMPT FOR SQL GENERATION AND VISUALIZATION GENERATION

Listing 1 shows the prompt template we use to generate SQL code from an NL question. The placeholders `{table_info}`, `{fewshot_demo}`, and `{question}` correspond to schema information, few-shot demonstrations, and the input NL question, respectively. A SQL query wrapper guides the LLM to produce structured output, enabling post-processing components to reliably extract the generated SQL query using regular expression matching.

Listing 2 shows the template used to format schema information. After the table name, the schema is presented in tabular form, where each row represents an attribute in the database. The columns `name`, `type`, `notnull`, `dflt_value`, and `pk` indicate the column name, declared data type, whether the column has a non-null constraint, its default value, and whether it is a primary key. Each table is formatted in this way, and tables are listed sequentially in lexicographical order.

Listing 3 shows the template for formatting few-shot demonstrations. After the demo NL question, we include a chain-of-thought step beginning with the phrase “Let’s think step-by-step,” followed by a list of relevant tables referenced in the demo SQL query. The demo SQL query itself is then presented. All few-shot demos in a prompt are displayed in this format and ordered by decreasing similarity of question embeddings, as described in the main text.

Listing 4 shows the prompt template we use to generate a visualization from an NL question and the corresponding extracted dataframe. The placeholders `{viz_name}`, `{columns}`, and `{question}` correspond to (1) the list of visualization types supported by our system’s front end (scatterplot, bar chart, line chart, and histogram), (2) the columns of the extracted dataframe, and (3) the input NL question, respectively.

Listing 1. Prompt template for SQL generation.

```
### You are a DuckDB expert.
Given an input question, write a syntactically correct DuckDB query that answers the input
question.
You must write the query using only functions and syntax supported by DuckDB.
Even though some examples below (the few-shot demos) include table names prefixed with
"physionet-data." and may use functions from other SQL dialects, your final query must:
- Use only DuckDB-compatible functions (for example, use DATEDIFF instead of TIMESTAMP_DIFF).
- Use local table names that are capitalized and do NOT include the "physionet-data." prefix.
- Never query for all columns from a table - only query the columns needed to answer the
  question.
- Wrap each column name in backticks (`) to denote them as delimited identifiers.
- Only reference column names that exist in the tables provided below.
- Be careful not to query for columns that do not exist, and ensure you are using the
  correct columns for each table.
- Use CURRENT_DATE() if the question involves "today".
- Exclude null values.
- Use aggregation functions like COUNT() and a GROUP BY clause if the question asks for
  distribution.
- Use different aliases for all tables and output columns.
Wrap the SQL query like this:
```sql
```

### Here is the information about the tables:
{schema_info}

### Some example pairs of questions and corresponding SQL queries (these examples might use
functions from other dialects) are provided below:
{fewshot_demo}

---

### Question: {question}
### SQL: Let’s think step-by-step.
```


Listing 2. Schema information for SQL generation for example table.

Table: admissions

Schema:

| | cid | name | type | notnull | dflt_value | pk |
|----|-----|--------------------|-----------|---------|------------|-------|
| 0 | 0 | row_id | BIGINT | False | None | False |
| 1 | 1 | subject_id | BIGINT | False | None | False |
| 2 | 2 | hadm_id | BIGINT | False | None | False |
| 3 | 3 | admittime | TIMESTAMP | False | None | False |
| 4 | 4 | disctime | TIMESTAMP | False | None | False |
| 5 | 5 | admission_type | VARCHAR | False | None | False |
| 6 | 6 | admission_location | VARCHAR | False | None | False |
| 7 | 7 | discharge_location | VARCHAR | False | None | False |
| 8 | 8 | insurance | VARCHAR | False | None | False |
| 9 | 9 | language | VARCHAR | False | None | False |
| 10 | 10 | marital_status | VARCHAR | False | None | False |
| 11 | 11 | age | BIGINT | False | None | False |

Listing 3. Example few-shot demo for SQL generation.

```
##
Question: What is the minimum total hospital cost that involved a procedure called other
enterostomy since 2100?
Answer: Let's think step-by-step.
1. Identify the relevant tables:
-- cost
-- procedures_icd
-- d_icd_procedures
2. Final SQL query:
SELECT MIN(T1.C1) FROM (SELECT SUM(cost.cost) AS C1 FROM cost WHERE cost.hadm_id IN (SELECT
procedures_icd.hadm_id FROM procedures_icd WHERE procedures_icd.icd_code = (SELECT
d_icd_procedures.icd_code FROM d_icd_procedures WHERE d_icd_procedures.long_title = 'other
enterostomy')) AND STRFTIME(CAST(cost.chargetime AS TIMESTAMP), '%Y') >= '2100' GROUP BY
cost.hadm_id) AS T1
```

Listing 4. Prompt template for visualization generation

```
### You are an expert in data visualization. Given an input question and the columns of a
dataframe, determine the type of visualization and the axis columns that most
appropriately address the question.
For the type of visualization, select only among the following options: {viz_names}. For the
axis columns, select only the column names you are given.
Select only one column if the visualization type requires only one column of data (e.g.,
histogram), and select two columns if the visualization type requires two columns of data
(e.g., scatterplot)
If the prompt is related to visualizing a distribution and there is a count column, choose the
bar chart and select two columns instead of selecting the histogram or density plot and
only one column.

Format the output like the following without any extra text or explanations (the Yaxis segment
can be omitted if only one column is selected):
VizType: VISUALIZATION-TYPE-NO; Xaxis: HORIZONTAL-AXIS; Yaxis: VERTICAL-AXIS
As an example, the output "VizType: 0; Xaxis: cal_daily; Yaxis: bmi" means a scatterplot with
the column "cal_daily" as the horizontal axis and the column "bmi" as the vertical axis.

### Here are the column names in the given dataframe.
{columns}

### Question: {question}
### Answer:
```

APPENDIX B MEDICAL LITERATURE–INSPIRED DEMOS

To supplement benchmark data, we created 105 few-shot demonstrations inspired by about 20 published studies that used the MIMIC-IV dataset. We extracted cohort selection criteria sections from the papers and adapted them into natural-language questions, including both direct adaptations and modifications to expand coverage across different database attributes and application scenarios. The corresponding gold SQL queries were first generated by `gpt-4-1106-preview` and then validated against DuckDB implementations of the MIMIC databases. We manually verified their correctness in our local database system, defined as valid, free of execution errors, and accurately presenting the information needed to address the corresponding question. Most of the debugging process consisted of resolving execution errors, but also included more arduous fixes, such as rewriting the entire query. Additional details are provided in the extended version available on arXiv: <https://arxiv.org/abs/2511.00772>.

APPENDIX C EHRSQL DATA POINT PREPROCESSING

We modified the train, validation, and test sets of EHRSQL, particularly the gold SQL queries, to ensure compatibility with CELEC’s design and execution environment. As CELEC operates on DuckDB instances of MIMIC-III and MIMIC-IV, we first applied the SQLGlot package in Python to translate gold SQL queries from SQLite into DuckDB syntax. Furthermore, we introduced a set of targeted preprocessing rules to address the most frequent issues SQLGlot failed to account for, namely current time and datetime expression errors. After translation, we filtered the training, validation, and test sets to retain only cases where the gold SQL executed successfully on the official MIMIC-IV demo database and returned a non-empty dataframe. This resulted in new dataset splits comprising 3,976 training queries, 785 validation queries, and 785 test queries. For system development, we further divided the new test set into a small validation portion of 78 queries (10%) for hyperparameter tuning and 707 queries (90%) for final evaluation, as described in the main text. Additional details are provided in the extended version available on arXiv: <https://arxiv.org/abs/2511.00772>.

APPENDIX D USER INTERFACE DEMONSTRATION

Additional UI demonstrations are provided in the extended version available on arXiv: <https://arxiv.org/abs/2511.00772>.